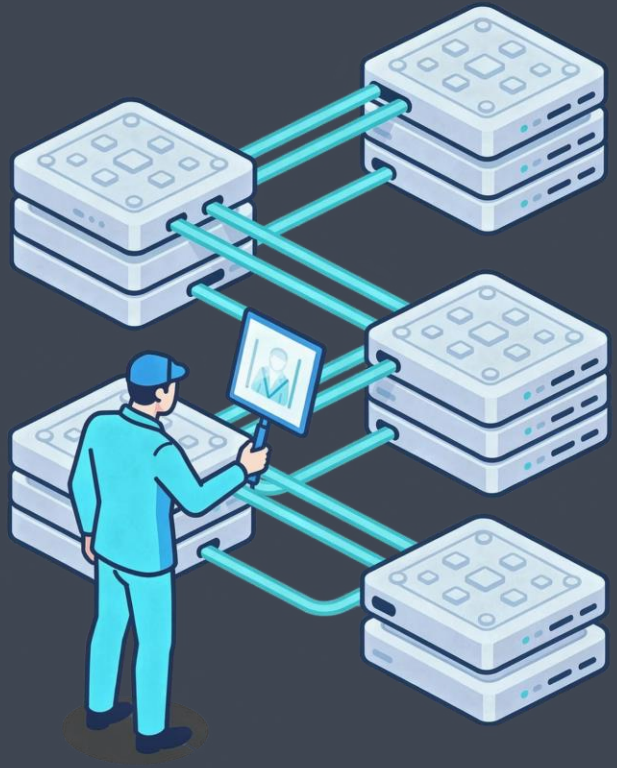


Volatile Memory Forensics

a possible example

Initial Triage — System Identification

Before deep analysis, anchor your investigation with basic facts from the image.

```
python3 vol.py -f memory.raw windows.info
python3 vol.py -f memory.raw
windows.registry.hivelist
```

OS Version

Does the reported build match the expected system?

Clock Consistency

Compare system time to acquisition timestamp — skew is suspicious.

Virtualisation

Look for VMware, Hyper-V, or VirtualBox artefacts in memory.

Process Analysis — Three Complementary Views

Each plugin walks a different data structure. Use all three together.

pslist

Walks the doubly-linked list at `PsActiveProcessHead`. Fast, but blind to unlinked processes.

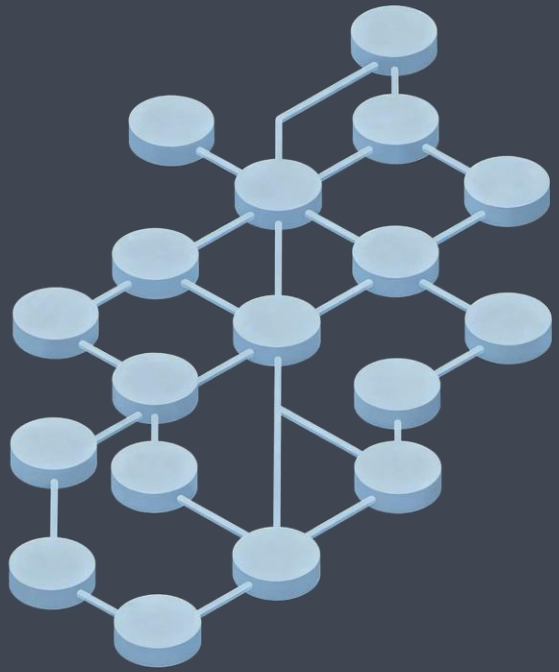
pstree

Traverses `EPROCESS.InheritedFromUniqueProcessId` to show parent-child hierarchy.

psscan

Scans raw memory for `Proc` pool tags. Finds hidden and unlinked processes that pslist misses.

⚠ **Key technique:** Compare pslist vs psscan output. A process present in psscan but absent from pslist indicates EPROCESS unlinking — a hallmark of rootkit activity.



Process Tree — Red Flags

Unexpected Parent-Child

`cmd.exe` or `powershell.exe` spawned by `winword.exe` or a browser - classic exploitation pattern.

Name Spoofing

`svchost.exe` running from `C:\Users\` rather than `C:\Windows\System32\` - path mismatch is an immediate IOC.

PPID Spoofing / Orphans

Process whose parent PID no longer exists - malware terminates its loader after injecting the child.

Timestamp Anomalies

Creation time inconsistent with system uptime or the surrounding process timeline.

DLL & Handle Analysis

DLL Inspection

```
windows.dlllist --pid 1234
windows.ldrmodules --pid 1234
```

`ldrmodules` cross-references three loader lists (`InLoadOrder`, `InMemOrder`, `InInitOrder`) against the VAD (Virtual Address Descriptor) tree. A DLL present in the VAD but absent from one or more lists signals injection or hollowing.

note: VAD is a per-process kernel data structure in Windows that describes the complete virtual address space layout of a process. Every region of virtual memory that a process has ever reserved or committed — executable images, heaps, stacks, mapped files, shared memory — has a corresponding VAD node describing it.

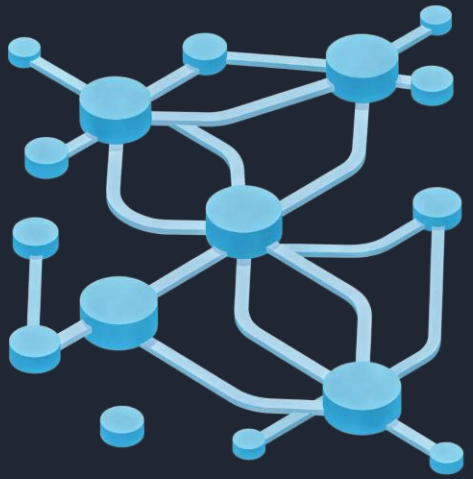
Handle Analysis

```
windows.handles --pid 1234
```

Enumerates open handles: files, registry keys, mutexes, and events. Key values:

- File handles expose what was open at dump time - even deleted files
- Mutex names are reliable IOCs - many malware families use unique mutexes to prevent re-infection

Network Analysis



```
windows.netstat
windows.netscan
```

`netscan` scans for TCP/UDP endpoint structures directly, surfacing connections that `netstat` may miss.

What to look for:

- Active C2 connections with remote IP and port
- Listening services on unexpected ports
- Recently closed connections still resident in memory
- The owning PID — linking network activity to a specific process

ⓘ An unexpected process — `explorer.exe`, a misplaced `svchost` — with an outbound connection on 443/80 is a common malware indicator

Code Injection Detection — malfind

```
python3 vol.py -f memory.raw windows.malfind
python3 vol.py -f memory.raw windows.memmap --pid 1234 --dump
```

Process Hollowing

Legitimate process created suspended; its memory replaced with malicious code.

Reflective DLL Injection

DLL loaded entirely from memory — no corresponding file on disk.

Shellcode Injection

Raw shellcode copied into a remote process's address space.

`malfind` flags VAD regions that are **executable**, **not file-backed**, and contain suspicious byte patterns (MZ headers, shellcode preambles). Output includes PID, virtual address, protection flags, and a hex+ASCII dump for immediate review.

Registry Analysis — In-Memory Hives

```
windows.registry.hivelist
windows.registry.printkey \
  --key "SOFTWARE\Microsoft\Windows\
CurrentVersion\Run"
windows.registry.hivescan
```

`hivescan` finds hive structures that are unloaded or hidden but still resident in memory pages.

What In-Memory Registry Recovers

Autorun Persistence

Malware-modified Run keys not yet visible on disk.

Unflushed Values

Recently written values that haven't been committed to the hive file.

Transiently Loaded Hives

Hives loaded temporarily (e.g., user profiles) and unloaded — but still in memory pages.

Artefact Extraction

```
windows.dumpfiles --pid 1234
windows.modules
windows.modscan
windows.dumpfiles --virtaddr [addr]
```

`modscan` finds hidden kernel drivers not present in the standard module list — a critical check for rootkits operating at ring 0.

Post-Extraction Workflow

1. **Submit to VirusTotal or a sandbox**

   Rapid triage against known malware signatures.

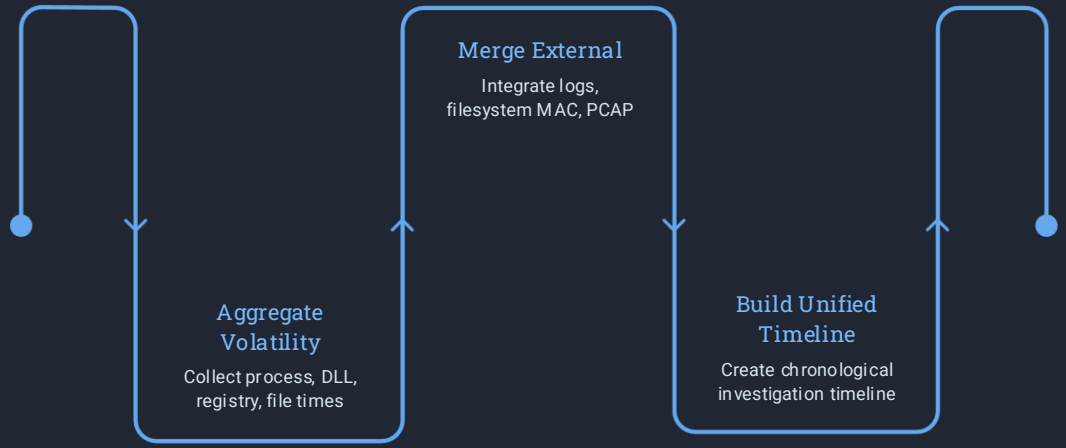
2. **Disassemble in Ghidra or IDA**

   Reverse-engineer functionality, identify C2 protocols, decode strings.

3. **Hash comparison**

   Diff extracted binaries against known-good hashes from a clean reference image.

Timeline Construction



A unified timeline is the connective tissue of the entire investigation — anchoring memory artefacts to real-world events.

```
python3 vol.py -f memory.raw
timeliner
```

`timeliner` aggregates timestamps from processes, loaded DLLs, registry keys, and file handles into a single chronological view.

Correlate With

- Windows Event Logs and Sysmon data
- Filesystem MAC times (MFT analysis)
- Network packet captures (PCAP)
- Proxy and DNS logs

Key Takeaways

Always Use Three Process Plugins

pslist + pstree + psscan. Discrepancies reveal rootkit hiding.

Loader List Cross-Reference

ldrmodules VAD mismatches are the clearest signal of DLL injection.

Network → Process Pivot

Unexpected outbound connections are often your fastest entry point to malware identification.

Memory Has No Secrets

Unflushed registry values, deleted file handles, and in-memory-only code all persist until overwritten.